

SISTEMAS OPERATIVOS FRENTE A CARGAS DE ALTO RENDIMIENTO

Videojuegos e Inteligencia Artificial como casos de estudio

Materia	Sistemas Operativos
Profesor	Gunnar Eyal Wolf Iszaavich
Integrantes	Hernández Moguel Francisco Itzcoatl Ajax Arcos Garduño
Grupo	07
Semestre	2027-1
Tema	Procesos, memoria, GPU, entrada/salida y concurrencia

Octubre de 2026

Resumen

Este trabajo analiza el papel del sistema operativo frente a cargas de alto rendimiento, utilizando videojuegos e inteligencia artificial como casos de estudio. El objetivo no es estudiar estas aplicaciones por sí mismas, sino observar cómo sus patrones de uso hacen visibles mecanismos fundamentales del sistema operativo: planificación de CPU, procesos e hilos, memoria virtual, acceso a GPU, entrada/salida y concurrencia. La investigación se apoya en bibliografía académica de sistemas operativos, documentación técnica oficial de Microsoft, Linux y NVIDIA, y utiliza Racing the Beam: The Atari Video Computer System como contraste histórico para mostrar cómo las restricciones de una plataforma condicionan la temporización gráfica, la administración de memoria y la relación entre software y hardware. Se presentan además tres ejemplos técnicos: competencia de hilos por CPU, generación de un frame bajo restricciones temporales y movimiento de datos hacia una GPU en una carga de IA.

Palabras clave: sistema operativo, planificación, hilos, memoria virtual, GPU, entrada/salida, videojuegos, inteligencia artificial, cargas de alto rendimiento.

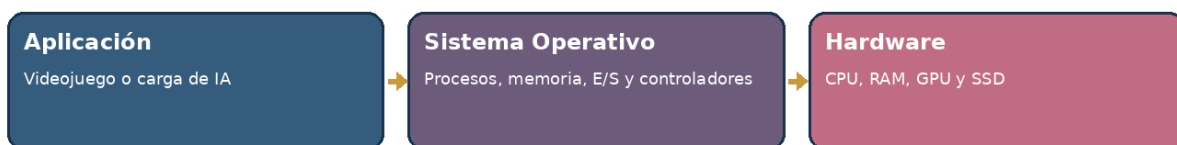
1. Introducción

Los sistemas operativos se suelen describir como la capa de software que administra los recursos del equipo y ofrece servicios a las aplicaciones. En una computadora moderna, ese trabajo implica repartir tiempo de procesador, aislar espacios de memoria, coordinar operaciones de entrada/salida y proporcionar mecanismos para trabajar con dispositivos especializados. Los textos clásicos de sistemas operativos organizan estas funciones alrededor de temas como virtualización, concurrencia, procesos, memoria y persistencia (Arpaci-Dusseau & Arpaci-Dusseau, 2023; Silberschatz, Galvin, & Gagne, 2018).

Las cargas de alto rendimiento son útiles como caso de estudio porque pueden exigir varios recursos al mismo tiempo. Un videojuego moderno puede combinar lógica, física, audio, renderizado, acceso a almacenamiento y respuesta interactiva; una carga de inteligencia artificial puede requerir grandes movimientos de datos, ejecución paralela en GPU y coordinación entre memoria principal y memoria del dispositivo. En ambos casos, el rendimiento observable no depende de una sola pieza de hardware, sino de la interacción entre aplicación, sistema operativo, controladores y dispositivos.

Por esta razón, la pregunta central de esta investigación es: ¿cómo administra un sistema operativo los recursos necesarios cuando una aplicación demanda cantidades importantes de cómputo, memoria y entrada/salida? La comparación entre videojuegos e IA permite responderla sin asumir que dichas aplicaciones requieren un sistema operativo diferente; más bien, permite observar cómo distintos patrones de carga ejercen presión sobre mecanismos comunes del sistema.

Relación entre aplicación, sistema operativo y hardware



2. Objetivos y metodología

Objetivo general

Analizar cómo los mecanismos de un sistema operativo intervienen en la administración de recursos frente a cargas de alto rendimiento, utilizando videojuegos e inteligencia artificial como casos de estudio.

Objetivos específicos

- Explicar la función de procesos, hilos y planificación de CPU.
- Describir el papel de la memoria virtual y la administración de memoria.
- Explicar la participación del sistema operativo y los controladores en el acceso a GPU.
- Distinguir entre entrada/salida síncrona y asíncrona.
- Comparar las necesidades de una carga interactiva de videojuegos con una carga de procesamiento de IA.
- Desarrollar tres ejemplos técnicos que integren los conceptos anteriores.
- Relacionar los mecanismos modernos con un antecedente histórico de plataforma de videojuegos mediante el caso del Atari VCS.

Metodología

Se realizó una investigación documental. La estructura temática parte de la exposición preparada por el equipo y se amplía con dos textos académicos de sistemas operativos, documentación técnica oficial de Microsoft, Linux Kernel y NVIDIA, y el libro *Racing the Beam: The Atari Video Computer System*, proporcionado por el profesor, como fuente complementaria para el análisis histórico de plataformas de videojuegos. La comparación es cualitativa: no se pretende demostrar que una categoría de aplicación sea “más rápida” que otra, sino identificar qué recursos, mecanismos y restricciones temporales son relevantes en cada patrón de carga.

3. Cargas de alto rendimiento y administración de recursos

Una carga de alto rendimiento puede exigir simultáneamente CPU, memoria, GPU y operaciones de entrada/salida. Esto no significa que todas las aplicaciones utilicen todos los recursos con la misma intensidad. Una aplicación puede estar limitada por cómputo, por memoria, por ancho de banda, por almacenamiento o por latencia. Desde el punto de vista del sistema operativo, el problema consiste en repartir recursos compartidos entre múltiples procesos y mantener abstracciones seguras y eficientes.

Los procesos no acceden de forma completamente directa a todos los recursos físicos. El sistema operativo crea abstracciones: procesos y hilos para la ejecución, espacios de direcciones virtuales para la memoria, archivos y dispositivos para la E/S y controladores para hardware especializado. Esta capa de administración permite que varias aplicaciones coexistan aun cuando compitan por CPU, RAM, GPU o almacenamiento.

4. Procesos, hilos y planificación de CPU

Un proceso es una instancia de un programa en ejecución y posee recursos como espacio de direcciones, identificadores y objetos del sistema. Dentro del proceso existen uno o más hilos. En Windows, el hilo es la entidad que puede ser planificada para ejecución; los hilos de un mismo proceso comparten el espacio de direcciones y varios recursos (Microsoft, s. f.-a).

El planificador de CPU decide qué hilos listos reciben tiempo de procesador. En Windows, la prioridad influye en esta decisión: el sistema mantiene hilos listos por niveles de prioridad y, cuando un procesador queda disponible, selecciona trabajo para ejecutar; si aparece un hilo de prioridad mayor puede producirse una sustitución del hilo en ejecución (Microsoft, s. f.-b). Por ello, que una aplicación cree muchos hilos no

garantiza que todos se ejecuten simultáneamente. En un sistema multinúcleo varios hilos pueden ejecutarse en paralelo, pero siguen compitiendo con otros procesos y servicios.

5. Videojuegos como carga interactiva

Un videojuego es una carga especialmente interesante porque combina trabajo de CPU y GPU con restricciones temporales perceptibles por el usuario. La CPU puede ocuparse de lógica, física, audio, red y preparación del renderizado, mientras que la GPU ejecuta operaciones gráficas. Al mismo tiempo, el juego debe acceder a texturas, modelos, mapas y otros recursos almacenados en memoria o disco.

La interacción entre CPU y GPU no debe describirse como si el sistema operativo ejecutara directamente la lógica del juego. La lógica pertenece a la aplicación o al motor; el sistema operativo proporciona planificación, memoria, E/S y acceso a dispositivos. En Direct3D 12, por ejemplo, la aplicación registra trabajo en command lists y las envía a command queues para su ejecución en GPU. Microsoft destaca que este modelo permite preparar trabajo desde varios hilos y aprovechar procesadores multinúcleo (Microsoft, s. f.-c).

Flujo simplificado de un frame

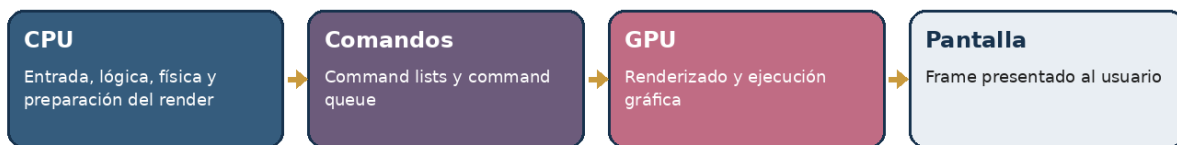


Figura 2. Flujo simplificado de trabajo CPU-GPU para producir un frame.

5.1 Contraste histórico: Atari VCS y temporización gráfica

El libro *Racing the Beam*, de Nick Montfort e Ian Bogost, estudia el Atari Video Computer System (VCS) como una plataforma cuyas restricciones técnicas influían directamente en la forma de programar. El VCS disponía de sólo 128 bytes de RAM y no contaba con memoria suficiente para mantener un framebuffer completo. En consecuencia, el programador debía producir la imagen línea por línea, sincronizando las instrucciones del procesador 6507 con el barrido del televisor mediante el Television Interface Adaptor (TIA). Esta necesidad de temporización estricta explica la idea de “racing the beam”: el código debía completar determinadas operaciones en ventanas de tiempo muy pequeñas para que la imagen se generara correctamente (Montfort & Bogost, 2009).

El contraste también resulta útil desde la perspectiva de Sistemas Operativos. El Atari VCS no tenía un sistema operativo encargado de interceptar entradas y proporcionar servicios generales como ocurre en una computadora moderna; incluso la respuesta a controles comunes debía implementarse en el propio programa del cartucho. En los sistemas actuales, muchas de esas responsabilidades se encuentran mediadas por el sistema operativo, sus controladores y sus abstracciones. Por ello, el caso del VCS no se utiliza aquí como equivalente de una PC moderna, sino como un antecedente que permite observar con claridad cómo la arquitectura de una plataforma determina qué trabajo debe asumir el software y qué restricciones temporales condicionan su rendimiento.

6. Administración de memoria

La memoria virtual permite que cada proceso opere con un espacio de direcciones propio, separado de la memoria física real. El sistema operativo mantiene las estructuras necesarias para traducir direcciones y proteger los espacios de distintos procesos. La paginación divide el espacio de memoria en unidades que pueden mapearse a memoria física de acuerdo con las necesidades del sistema (Arpaci-Dusseau & Arpaci-Dusseau, 2023).

En una carga de videojuegos, la memoria puede contener código, estado del mundo, texturas, audio y buffers. En una carga de IA puede contener datos de entrada, parámetros del modelo y resultados intermedios. En ambos casos, el problema no se reduce a la cantidad total de RAM disponible; también importa cómo se asigna, protege y mueve la información.

7. GPU y participación del sistema operativo

Las GPU modernas son dispositivos especializados con memoria y colas de trabajo propias, pero no funcionan completamente aisladas del sistema operativo. En Windows, el Video Memory Manager (VidMm) es un componente del kernel gráfico de DirectX encargado de administrar recursos de memoria de GPU, mientras que el GPU Scheduler (VidSch) participa en la planificación del trabajo gráfico (Microsoft, s. f.-d). WDDM también incorpora mecanismos de memoria virtual de GPU para manejar espacios de direcciones utilizados por motores gráficos (Microsoft, s. f.-e).

En Linux, Heterogeneous Memory Management (HMM) proporciona infraestructura para integrar memoria de dispositivos, como la memoria de una GPU, dentro de los mecanismos normales de memoria del kernel y facilitar modelos de memoria compartida entre CPU y dispositivo (Linux Kernel Documentation, s. f.). Esto muestra que el soporte a aceleradores involucra mecanismos concretos del sistema operativo y de sus controladores.

8. Inteligencia artificial y cómputo heterogéneo

Muchas cargas modernas de inteligencia artificial utilizan un modelo heterogéneo: la CPU controla el flujo general y prepara datos, mientras una GPU u otro acelerador realiza grandes cantidades de operaciones paralelas. El modelo de programación CUDA distingue una jerarquía de hilos y varios espacios de memoria, incluyendo memoria local, compartida y global (NVIDIA, s. f.).

Desde el punto de vista del sistema operativo, es importante considerar el camino de los datos. Una imagen o lote de datos puede comenzar en almacenamiento, pasar a memoria principal, ser preparado por la CPU y finalmente transferirse a memoria del dispositivo. Si el acelerador termina esperando a que lleguen los datos, el cuello de botella deja de ser puramente computacional. Por ello, memoria, E/S y sincronización son tan relevantes como la capacidad de cálculo de la GPU.

Ruta simplificada de datos en una carga de IA

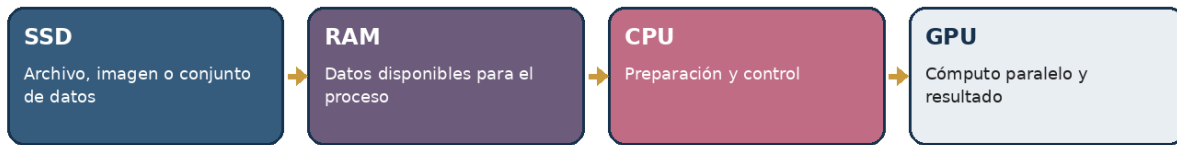


Figura 3. Ruta simplificada de los datos en una carga de IA.

9. Entrada/salida y concurrencia

Las operaciones de entrada/salida pueden ser síncronas o asíncronas. En E/S síncrona, un hilo inicia una operación y espera hasta que termina. En E/S asíncrona, el hilo puede emitir la solicitud y continuar realizando otro trabajo mientras el kernel procesa la operación; posteriormente recibe una señal o notificación de finalización (Microsoft, 2025).

Este mecanismo es útil para comprender por qué una aplicación interactiva intenta evitar esperas innecesarias. Un videojuego puede solicitar la lectura de una textura sin detener todo el procesamiento del frame, y una aplicación de IA puede leer datos o modelos mientras otros recursos continúan activos. La E/S asíncrona no elimina el costo del almacenamiento, pero permite superponer trabajo y reducir periodos de inactividad del hilo solicitante.

10. Comparación de patrones de carga

Videojuegos e inteligencia artificial utilizan recursos similares, pero no necesariamente los priorizan de la misma manera. Una carga interactiva suele ser sensible a la latencia y a la regularidad temporal; una carga de entrenamiento de IA puede priorizar throughput sostenido, mientras que una carga de inferencia también puede ser sensible a latencia. Por ello, no es correcto reducir la comparación a “videojuegos usan GPU para gráficos” e “IA usa GPU para cálculo”. La diferencia más interesante está en el patrón de utilización y en las restricciones temporales de cada aplicación.

Tabla comparativa

Categoría	Videojuegos	Inteligencia artificial
Tipo de carga	Interactiva	Procesamiento / inferencia
CPU	Lógica, simulación y preparación	Preparación y control
GPU	Renderizado	Cómputo paralelo
Memoria	Recursos del mundo y buffers	Modelos, datos y resultados
E/S	Streaming de recursos	Datasets, modelos y archivos
Sensibilidad	Latencia y estabilidad temporal	Throughput o latencia, según la aplicación

11. Tres ejemplos técnicos integradores

11.1 Competencia de hilos por CPU

Supóngase un videojuego con trabajo relacionado con lógica, física, audio y preparación del renderizado. Al mismo tiempo están activos un navegador, un cliente de comunicación y servicios del sistema. Todos estos procesos contienen hilos que pueden pasar al estado listo. El planificador debe seleccionar cuáles se ejecutan en los procesadores disponibles. Si un hilo crítico para el siguiente frame no obtiene CPU a tiempo, aumenta la latencia de esa etapa. El ejemplo muestra que el rendimiento no depende solamente de “tener muchos núcleos”, sino de cómo se distribuye el trabajo y de qué otros hilos compiten por CPU.

11.2 Un frame contra el reloj

En una pantalla de 60 Hz existe una oportunidad de actualización aproximadamente cada 16.67 ms. Durante ese intervalo, la aplicación debe procesar entrada, actualizar lógica y física, preparar comandos y coordinar el renderizado en GPU. Un frame puede tardar más que los demás por contención de CPU, espera de E/S, compilación de recursos, sincronización o carga gráfica. Aunque el promedio de FPS sea aceptable, un pico aislado de frame time puede percibirse como stutter. Este ejemplo integra planificación, CPU, GPU, memoria y latencia.

11.3 IA: cuando el cuello de botella no es el cálculo

Considérese una aplicación que clasifica una imagen almacenada en SSD. El archivo debe leerse, quedar disponible en RAM, ser preparado por la CPU y transferirse a la memoria accesible por la GPU antes de ejecutar el cómputo. Si la GPU termina su trabajo anterior y los siguientes datos todavía no están disponibles, queda ociosa. El límite del rendimiento se desplaza entonces hacia almacenamiento, memoria o transferencia, aunque la GPU sea muy potente. Este ejemplo integra E/S, memoria, CPU, GPU y movimiento de datos.

12. Discusión

Los tres ejemplos muestran que una carga de alto rendimiento expone decisiones que normalmente permanecen ocultas para el usuario. La planificación determina cuándo se ejecuta un hilo; la memoria virtual organiza y protege datos; la E/S condiciona cuándo están disponibles los recursos; y los mecanismos del subsistema gráfico coordinan memoria y trabajo de GPU. Ninguno de estos componentes explica por sí solo el rendimiento final.

También es importante separar responsabilidades. El sistema operativo no realiza directamente la lógica de un videojuego ni las operaciones matemáticas de una red neuronal. Esas tareas pertenecen a la aplicación, bibliotecas y aceleradores. El papel del sistema operativo es proporcionar mecanismos de ejecución, memoria, aislamiento, comunicación y acceso a dispositivos. Esta distinción evita atribuir al kernel funciones que corresponden al motor del juego, a una API gráfica o a un framework de IA.

Finalmente, la comparación muestra que usar los mismos recursos no implica perseguir los mismos objetivos. Una carga interactiva puede valorar especialmente la predictibilidad temporal, mientras una carga de procesamiento puede buscar mantener ocupado el hardware durante periodos prolongados. La labor del sistema operativo consiste en administrar recursos compartidos de manera general, mientras aplicaciones y bibliotecas construyen encima políticas específicas de rendimiento. El contraste con el Atari VCS refuerza esta idea: cuando la plataforma ofrece pocas abstracciones, el programa debe asumir directamente más tareas de temporización y control del hardware; en sistemas modernos, una parte importante de esa complejidad se distribuye entre el sistema operativo, los controladores, las bibliotecas y la propia aplicación.

13. Conclusiones

Los videojuegos y la inteligencia artificial son aplicaciones, pero sus patrones de carga permiten observar de forma clara funciones esenciales del sistema operativo. En ambos casos aparecen procesos e hilos, planificación de CPU, memoria virtual, entrada/salida y acceso a GPU.

La principal diferencia no está en que cada aplicación utilice recursos completamente distintos, sino en cómo los utiliza y qué restricciones predominan. Los videojuegos suelen ser sensibles a latencia y estabilidad temporal; las cargas de IA pueden priorizar throughput, capacidad de procesamiento o latencia de inferencia. En consecuencia, una evaluación correcta debe considerar el patrón de carga y no únicamente el porcentaje de uso de CPU o GPU.

El sistema operativo actúa como una capa de administración que permite compartir recursos, proteger memoria, coordinar dispositivos y mantener múltiples aplicaciones activas. Su influencia sobre el rendimiento se vuelve especialmente visible cuando CPU, memoria, GPU y almacenamiento deben trabajar de forma coordinada bajo una carga exigente. El caso histórico presentado en *Racing the Beam* muestra además que esta relación entre restricciones de hardware, organización de memoria y exigencias temporales no es nueva: cambia la tecnología, pero sigue siendo necesario comprender la plataforma completa para explicar el rendimiento observable.

Referencias

- Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2023). *Operating Systems: Three Easy Pieces* (Version 1.10). Arpaci-Dusseau Books. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.
- Montfort, N., & Bogost, I. (2009). *Racing the Beam: The Atari Video Computer System*. MIT Press.
- Microsoft. (s. f.-a). About Processes and Threads. Microsoft Learn. <https://learn.microsoft.com/windows/win32/procthread/about-processes-and-threads>
- Microsoft. (s. f.-b). Scheduling Priorities. Microsoft Learn. <https://learn.microsoft.com/windows/win32/procthread/scheduling-priorities>
- Microsoft. (s. f.-c). Work submission in Direct3D 12. Microsoft Learn. <https://learn.microsoft.com/windows/win32/direct3d12/command-queues-and-command-lists>
- Microsoft. (s. f.-d). Video Memory Management and GPU Scheduling. Microsoft Learn. <https://learn.microsoft.com/windows-hardware/drivers/display/video-memory-management-and-gpu-scheduling>
- Microsoft. (s. f.-e). GPU virtual memory in WDDM 2.0. Microsoft Learn. <https://learn.microsoft.com/windows-hardware/drivers/display/gpu-virtual-memory-in-wddm-2-0>
- Microsoft. (2025). Synchronous and Asynchronous I/O. Microsoft Learn. <https://learn.microsoft.com/windows/win32/fileio/synchronous-and-asynchronous-i-o>
- Linux Kernel Documentation. (s. f.). Heterogeneous Memory Management (HMM). <https://kernel.org/doc/html/latest/mm/hmm.html>
- NVIDIA. (s. f.). CUDA C++ Programming Guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>